Searching in Graphs

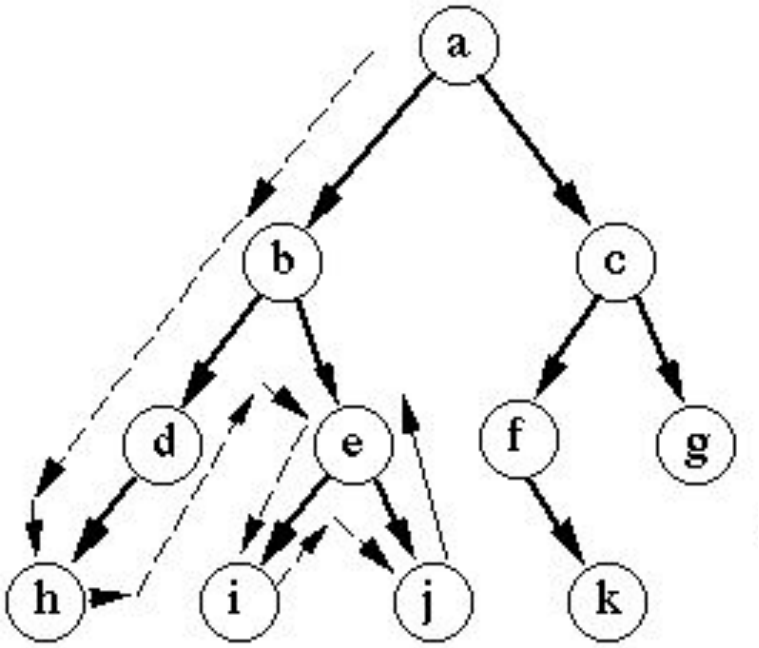
By: Guillermo Andres De Mendoza Corrales

DFS

Depth-First Search

Concept: DFS is an "uninformed" search that explores a graph by going as deep as possible along each branch before backtracking. Imagine walking through a maze and always following the left wall until you hit a dead end, then turning back to the last junction.

- **Mechanism:** Uses a **Stack** (LIFO - Last In, First Out) or **Recursion**.
- **Behavior:** It prioritizes "depth" over "breadth."
- **Key Characteristic:** It does **not** guarantee the shortest path. It only guarantees finding *a* path if one exists.

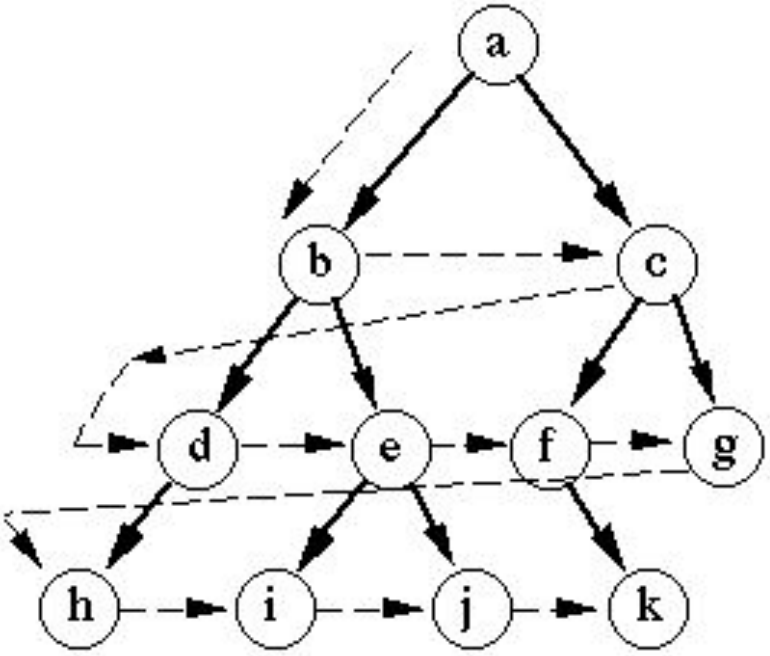


BFS

Breadth-First Search (BFS)

Concept: BFS is an "uninformed" search that explores all neighbor nodes at the present depth before moving on to the nodes at the next depth level. It radiates outward from the starting point like a ripple in a pond.

- **Mechanism:** Uses a **Queue** (FIFO - First In, First Out).
- **Behavior:** It explores level by level.
- **Key Characteristic:** In an unweighted graph (like a simple maze), BFS **always finds the shortest path** in terms of the number of edges.

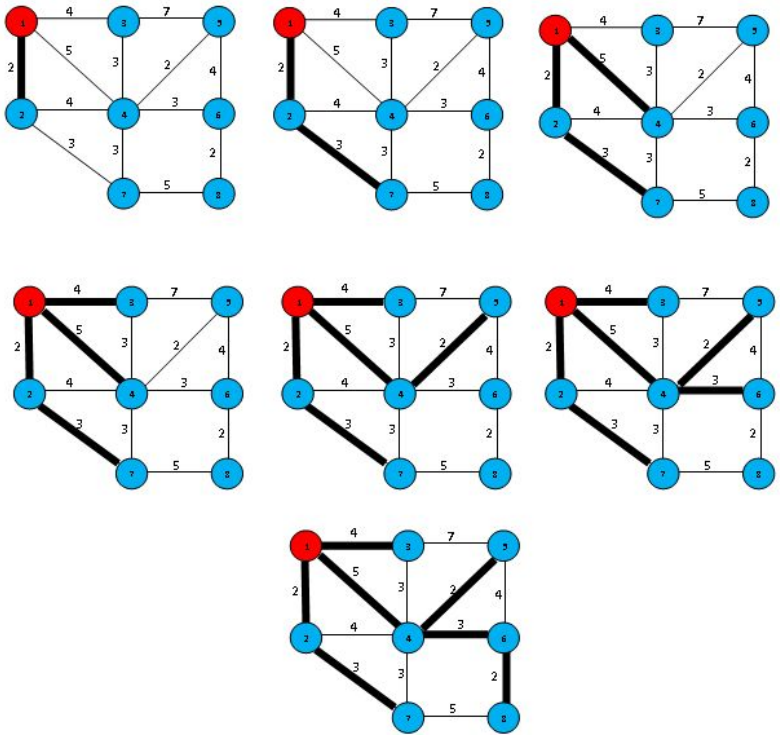


Dijkstra’s

Dijkstra’s Algorithm

Concept: Dijkstra is a "greedy" search used to find the shortest path between nodes in a **weighted graph**. It keeps track of the "current shortest distance" from the start to every other node and always chooses the unvisited node with the smallest distance to explore next.

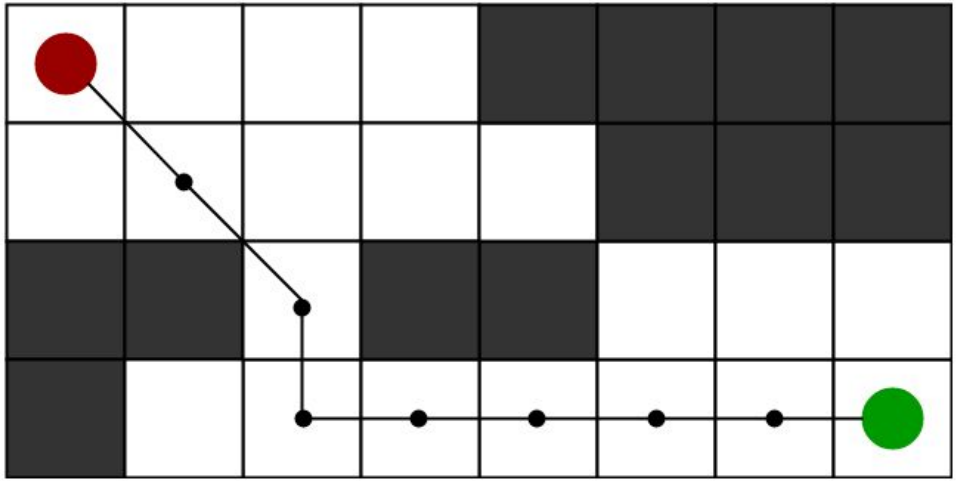
- **Mechanism:** Uses a **Priority Queue** (Min-Priority).
- **Behavior:** It accounts for edge costs (e.g., some paths in the maze might be "muddy" and slower than others).
- **Key Characteristic:** It is optimal for weighted graphs but fails if there are negative edge weights (not an issue in mazes).
- **Complexity:** O((V + E) \log V) with a binary heap.



A* Search (A-Star)

Concept: A* is an "informed" search (or heuristic search). It improves upon Dijkstra by using extra information about the goal's location. It doesn't just look at how far it has traveled (g); it also guesses how far is left to go (h).

- **The Formula:** f(n) = g(n) + h(n)
  - g(n): Actual cost from the start to the current node.
  - h(n): **Heuristic** (estimated cost) from the current node to the goal.
- **Mechanism:** Uses a **Priority Queue** based on the value of f(n).
- **Key Characteristic:** It is much faster than BFS or Dijkstra in large maps because it "aims" towards the target, avoiding unnecessary exploration in the opposite direction.
- **Heuristic for Mazes:** Usually the **Manhattan Distance** (the sum of vertical and horizontal steps).



Comparison

Algorithm	Data Structure	Best For...	Shortest Path?
DFS	Stack	fast path	No
BFS	Queue	Unweighted shortest path	Yes
Dijkstra	Priority Queue	Weighted shortest path	Yes
A*	Priority Queue	Efficiency using a target (Heuristic)	Yes (if h is admissible)